

Name : P. Yaswanth

Reg No : 192524167

Experiment: Depth-First Search (DFS)

Aim

To implement the **Depth-First Search (DFS)** algorithm using Python to traverse a graph.

Objectives

- To understand the Depth-First Search (DFS) algorithm.
- To implement DFS using Python.
- To traverse all vertices of a graph using recursion.
- To understand the concept of graph traversal.
- To analyze the traversal order of DFS.

Algorithm

1. Create a graph using an adjacency list.
2. Select the starting vertex.
3. Mark the starting vertex as visited.
4. Visit the current vertex and display it.
5. Recursively visit each unvisited adjacent vertex.
6. Continue until all reachable vertices are visited.

Python Program

```
# DFS Function
```

```
def dfs(graph, vertex, visited):
```

```
    if vertex not in visited:
```

```
        print(vertex, end=" ")
```

```
        visited.add(vertex)
```

```
        for neighbor in graph[vertex]:
```

```
dfs(graph, neighbor, visited)
```

```
# Graph Representation
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

```
visited = set()
```

```
print("DFS Traversal:")
```

```
dfs(graph, 'A', visited)
```

Sample Output

DFS Traversal:

A B D E F C

Out Put

```
1 # DFS Function
2 def dfs(graph, vertex, visited):
3     if vertex not in visited:
4         print(vertex, end=" ")
5         visited.add(vertex)
6
7     for neighbor in graph[vertex]:
8         dfs(graph, neighbor, visited)
9
10 # Graph Representation
11 graph = {
12     'A': ['B', 'C'],
13     'B': ['D', 'E'],
14     'C': ['F'],
15     'D': [],
16     'E': ['F'],
17     'F': []
18 }
19
20 visited = set()
21
22 print("DFS Traversal:")
23 dfs(graph, 'A', visited)
```

DFS Traversal:
A B D E F C
=== Code Execution Successful ===

Result

The Depth-First Search (DFS) algorithm was successfully implemented in Python. The program traversed all the vertices of the graph starting from the given source vertex.

Conclusion

The Depth-First Search (DFS) algorithm explores each branch of a graph as deeply as possible before backtracking. It uses **recursion** (or a stack) to manage the traversal process and ensures that each vertex is visited only once. DFS is widely used in Artificial Intelligence, graph traversal, path finding, cycle detection, and topological sorting.